

MBLP136 – VERİ TABANI

4. Hafta: Kütüphane Yönetim Sistemi Fiziksel Tasarımı

Dr. Öğr. Üyesi Hasan Oğuz

10 Mart 2026

İstanbul Okan Üniversitesi
Meslek Yüksekokulu

SQL Komut Grupları: DDL ve DML

Veritabanı işlemleri, amaçlarına göre dört ana gruba ayrılır. Tasarımcı olarak en çok **DDL** ve **DML** yapılarını kullanacağız:

- **DDL (Data Definition Language):** Veritabanı nesnelerini (tablo, indeks, view) oluşturur ve yapısını tanımlar.
- **DML (Data Manipulation Language):** Tablo içindeki verilerle etkileşime girer (seçme, ekleme, silme).
- **DCL & TCL:** Yetkilendirme ve işlem (transaction) kontrolü süreçlerini yönetir.

Temel Komut Matrisi (2/2)

Kategori	Komut	İşlev
DDL	<i>CREATE / ALTER / DROP</i>	Tablo oluşturur, günceller veya siler.
DML	<i>SELECT / INSERT</i>	Veri sorgular ve yeni kayıt ekler.
DML	<i>UPDATE / DELETE</i>	Mevcut veriyi günceller veya siler.
TCL	<i>COMMIT / ROLLBACK</i>	İşlemleri onaylar veya geri alır.

Kritik Fark

DROP tabloyu tamamen yok ederken, *DELETE* tablonun içindeki satırları siler fakat tablo yapısını korur.

Veri Tipleri ve Bellek Yönetimi

Veritabanı performansını optimize etmek için doğru karakter setini seçmek şarttır:

- **CHAR(*n*)**: Sabit uzunluk. Veri kısa olsa bile *n* kadar yer kaplar (Örn: TC No).
- **VARCHAR(*n*)**: Değişken uzunluk. Sadece girilen karakter kadar yer kaplar.
- **NVARCHAR(*n*)**: Unicode desteği. Türkçe karakterlerin (ğ, ş, İ) bozulmadan saklanması için **zorunludur**.

Öneri

Ad, Soyad ve Adres gibi alanlarda her zaman **NVARCHAR** tercih edilmelidir.

Sayısal Tipler:

- *INT*: Tam sayılar için.
- *DECIMAL(p, s)*: Para birimi ve hassas ölçümler için.
- *FLOAT*: Bilimsel hesaplamalar.

Zaman Tipleri:

- *DATE*: Sadece tarih.
- *DATETIME*: Tarih + Saat.
- *TIME*: Sadece saat.

Veri Bütünlüğü Kısıtlamaları (Constraints)

Verinin doğruluğunu yazılım katmanında değil, veritabanı katmanında garanti altına alıyoruz:

- **PRIMARY KEY (PK):** Satırı tekilleştirir. Boş (*NULL*) olamaz.
- **UNIQUE:** Sütundaki değerlerin benzersiz olmasını sağlar (Örn: E-posta, Telefon).
- **IDENTITY:** Sayısal anahtarların otomatik artmasını sağlar.

- **FOREIGN KEY (FK):** İki tablo arasındaki ilişkiyi kurar. "Referansal Bütünlük" sağlar.
- **CHECK:** Sütuna girilecek veriyi bir mantık süzgecinden geçirir (Örn: *Fiyat* > 0).
- **DEFAULT:** Değer girilmediğinde atanacak sabit değerdir (Örn: Tarih için *GETDATE()*).

Referansal Bütünlük

FK kısıtlaması sayesinde, kütüphanede kayıtlı olmayan bir kitaba ödünç işlemi yapılması sistem tarafından otomatik olarak engellenir.

Sistem Mimarisi ve Veritabanı Oluřturma

Veritabanı Konteynırı (01_CreateDatabase.sql)

Bir sistem tasarlarken ilk adım, verinin içinde yaşayacağı uzayı (namespace) tanımlamaktır.

```
-- SQL Server Management Studio (SSMS) için ıveritabanı şolutorma ğbetii
IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'Kutuphane') -- Kutuphane DB var ım?
BEGIN
    CREATE DATABASE Kutuphane; -- Yoksa Kutuphane isimli bir DB ıyaratır
END
GO

USE Kutuphane; -- Varsa direkt Kutuphane DB'ini seçer
GO
```

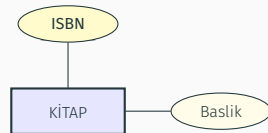
Mantıksal Not

IF NOT EXISTS kontrolü, scriptin tekrar tekrar çalıştırılabilir (idempotent) olmasını sağlar; bu, büyük ölçekli sistem yönetiminde kritik bir "best practice"tir.

Varlık Tanımlama: Kitaplar ve Üyeler

Kitap Varlığı ve Kısıtlamalar (02_Table_Kitaplar.sql)

```
CREATE TABLE Kitaplar (  
    ISBN NVARCHAR(20) PRIMARY KEY, -- Benzersiz anahtar  
    Baslik NVARCHAR(255) NOT NULL, -- Bos gecilemez  
    Yazar NVARCHAR(255),  
    -- Sayfa sayisinin pozitif olma zorunlulugu:  
    SayfaSayisi INT CONSTRAINT CHK_SayfaSayisi CHECK (SayfaSayisi > 0),  
    YayinTarihi DATE,  
    Fiyat DECIMAL(10, 2),  
    EklenmeTarihi DATETIME DEFAULT GETDATE() -- Otomatik zaman idamgas  
);  
  
INSERT INTO Kitaplar (ISBN, Baslik) VALUES ('978-605', 'Veri Tabani Sistemleri');
```



PK ve Check: ISBN veriye hızlı erişimi (Clustered Index) sağlarken, CHECK kısıtlaması fiziksel gerçekliği (negatif sayfa olamaz) korur.

Üye Yönetimi ve Veri Tipleri (03_Table_Uyeler.sql)

```
CREATE TABLE Uyeler (  
    UyeID INT IDENTITY(1,1) PRIMARY KEY, -- 1'den baslar, 1'er artar  
    TCKimlikNo NCHAR(11) UNIQUE NOT NULL, -- Sabit 11 karakter ve benzersiz  
    Ad NVARCHAR(100) NOT NULL,  
    Soyad NVARCHAR(100) NOT NULL,  
    Eposta NVARCHAR(255) UNIQUE,  
    Telefon NVARCHAR(20),  
    KayitTarihi DATETIME DEFAULT GETDATE(),  
    UyelikDurumu BIT DEFAULT 1 -- 1: Aktif, 0: Pasif  
);
```

NCHAR vs NVARCHAR

TC Kimlik No gibi uzunluğu değişmeyen verilerde **NCHAR** kullanmak, SQL Server'ın bellek yönetimini optimize eder. Değişken uzunluklu (Ad/Soyad) verilerde ise **NVARCHAR** ile disk alanından tasarruf edilir.

İlişkisel Simetri: Ödünç İşlemleri

Bağlantı Tablosu ve Foreign Key (04_Table_OduncIslemleri.sql)

Kitaplar ve Üyeler arasındaki M : N (Çoktan-Çoğa) ilişkiyi çözen yapı:

```
CREATE TABLE OduncIslemleri (  
    IslemID INT IDENTITY(1,1) PRIMARY KEY,  
    ISBN NVARCHAR(20), -- Kitaplar tablosuna referans  
    UyeID INT,          -- Uyeler tablosuna referans  
    VerilisTarihi DATETIME DEFAULT GETDATE(),  
    TeslimTarihi DATETIME,  
    -- Hesaplanan sutun: Iade suresi otomatik 15 gun eklenir  
    SonTeslimTarihi AS DATEADD(day, 15, VerilisTarihi),  
  
    CONSTRAINT FK_Kitap FOREIGN KEY (ISBN) REFERENCES Kitaplar(ISBN),  
    CONSTRAINT FK_Uye FOREIGN KEY (UyeID) REFERENCES Uyeler(UyeID)  
);
```


Filo Takip: Ortam Hazırlığı

Veritabanı Oluşturma ve Bağlam (01_Setup)

Sistem tasarımında ilk adım, verinin fiziksel olarak saklanacağı alanı tanımlamak ve operasyonel bağlamı (context) belirlemektir.

```
IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'FiloTakip')
BEGIN
    CREATE DATABASE FiloTakip; -- Veri uzayini rezerve eder
END
GO
USE FiloTakip; -- Sorgu baglamini degistirir
GO
```

- **Idempotency:** *IF NOT EXISTS* kontrolü, scriptin hata vermeden tekrar çalıştırılabilmesini sağlar.
- **Bağlam Geçişi:** *USE* komutu, sonraki tabloların yanlışlıkla *master* veritabanına kurulmasını engeller.

Varlık Yönetimi

Araç Varlığı ve Fiziksel Kısıtlamalar (02_Araclar)

Araçlar, sistemin ana varlığıdır (*Entity*). Tasarımda hem doğal hem de mantıksal kısıtlamalar kullanılmıştır:

```
CREATE TABLE Araclar (  
    Plaka NVARCHAR(15) PRIMARY KEY, -- Doğal Anahtar  
    Yil INT CHECK (Yil > 1950),      -- Mantıksal Sinir  
    SasiNo NVARCHAR(17) UNIQUE NOT NULL, -- Tekil Kimlik  
    Durum BIT DEFAULT 1              -- Boole Mantığı (0/1)  
);
```

Teknik Analiz

- **CHECK:** $Yil > 1950$ koşulu, veritabanı seviyesinde bir "Invariant" tanımlayarak kirli veri girişini engeller.
- **BIT:** Durum alanı, bellek optimizasyonu sağlayarak aracın müsaitlik durumunu flag mekanizmasıyla yönetir.

Sistemin diğer bileşenleri, otomatik artan anahtarlar (*Surrogate Keys*) ile tanımlanmıştır:

Aksesuarlar:

```
AksesuarID INT IDENTITY(1,1) PRIMARY KEY,  
AksesuarAdi NVARCHAR(100)
```

Müşteriler:

```
MusteriID INT IDENTITY(1,1),  
EhliyetNo NVARCHAR(20) UNIQUE
```

Neden UNIQUE?

EhliyetNo üzerine konulan **UNIQUE** kısıtlaması, yasal uyumluluk gereği bir kişinin sistemde mükerrer kayıt oluşturmalarını engeller.

İlişkisel Haritalama

Çoka-Çok (M : N) İlişki Çözümü (04_AracAksesuar)

Bir araçta birçok aksesuar olabilir, bir aksesuar tipi birçok araca takılabilir. Bu simetriyi bozmak için **Junction Table** kullanılır:

```
CREATE TABLE AracAksesuar (  
    Plaka NVARCHAR(15),  
    AksesuarID INT,  
    PRIMARY KEY (Plaka, AksesuarID), -- Birlesik Anahtar  
    FOREIGN KEY (Plaka) REFERENCES Araclar(Plaka),  
    FOREIGN KEY (AksesuarID) REFERENCES Aksesuarlar(AksesuarID)  
);
```

- **Composite PK:** İki yabancı anahtarın birleşimi, aynı aksesuarın aynı araca mükerrer eklenmesini önler.
- **Referansal Bütünlük:** Olmayan bir araç plakasına aksesuar atanamaz.

İşlemsel Süreçler

Kiralama Hareketleri (05_Kiralamalar)

Sistemin dinamik verisi, zaman damgalı bu tabloda tutulur:

```
CREATE TABLE Kiralamalar (  
  AlisTarihi DATETIME DEFAULT GETDATE(),  
  IadeTarihi DATETIME, -- Henuz donmediyse NULL  
  CONSTRAINT FK_Arac FOREIGN KEY (Plaka) REFERENCES Araclar(Plaka)  
);
```

Mantıksal Durum Takibi

- **NULL Mantığı:** *IadeTarihi* alanının **NULL** olması, kiralama işleminin aktif (araç yolda) olduğunu gösterir.
- **GETDATE():** İşlem anındaki sistem saatini otomatik kaydederek veri giriş hatasını minimize eder.

Yabancı Anahtar (FK) Mekanizması

Neden **CONSTRAINT** kullanıyoruz?

- **Orphan Records (Yetim Kayıtlar):** Silinmiş bir müşteriye ait kiralama kaydının kalmasını engeller.
- **Cascade Etkisi:** Ana tabloda yapılan güncellemelerin ilişkili tablolara yansımalarını kontrol eder.



Sonuç: Veritabanı sadece veri

saklamaz, verinin birbiriyle olan mantıksal bağıını korur.

Örnek Proje: Spor Salonu Takip Sistemi

Spor salonu yönetimi, statik üye verisi ile dinamik log verisinin senkronizasyonunu gerektirir:

- **Personel Yönetimi:** Eğitimcilerin ve idari personelin yetkinlik bazlı takibi.
- **Paket Normalizasyonu:** Üyelik tiplerinin (*Gold, Silver*) ayrı tabloda tutulması, fiyat güncellemelerinin tüm üyeleri otomatik etkilemesini sağlar.
- **Hareket Takibi:** Giriş-çıkış logları, tesisin yoğunluk analizini yapmamıza olanak tanır.

Zaman Serisi Verisi ve Kısıtlamalar (2/2)

Giriş-çıkış verilerinde veri bütünlüğünü sağlamak için fiziksel kısıtlamalar (*Constraints*) hayati önem taşır:

```
-- Zaman siralamasi kontrolü (Temporal Integrity)
CONSTRAINT CHK_Zaman CHECK (CikisZamani > GirisZamani)

-- Ölçeklenebilirlik için veri tipi seçimi
LogID BIGINT IDENTITY(1,1) -- Milyonlarca ıkayt kapasitesi
```

Mantıksal Analiz

Antrenman süresinin (Δt) hesaplanması, sistemin verimlilik metriğidir:

$$\Delta t = T_{\text{çıkış}} - T_{\text{giriş}}$$

Bu veri, salonun hangi saatlerde "kritik yoğunluğa" ulaştığını belirlemek için kullanılır.

Uygulama: Spor Salonu Takip Sistemi

Spor salonu yönetimi, statik üye verisi ile yüksek frekanslı giriş-çıkış verisinin senkronizasyonunu gerektirir:

- **Paket Normalizasyonu:** Üyelik tiplerinin ayrı tabloda tutulması, fiyat güncellemelerinin tüm üyeleri merkezi olarak etkilemesini sağlar.
- **Temporal Integrity (Zaman Bütünlüğü):** *CHECK* kısıtlaması ile çıkış saatinin girişten önce olması fiziksel olarak engellenir.
- **BIGINT Kullanımı:** Milyonlarca giriş kaydı potansiyeline karşı *INT* yerine *BIGINT* (8-byte) tercih edilmiştir.

Özet ve Değerlendirme

Haftalık Özet: Üç Farklı Model, Tek Mantık

Bu hafta geliştirdiğimiz üç ana mimariyi karşılaştıralım:

Senaryo	İlişki Tipi	Kritik Veri Tipi	Odak Noktası
Kütüphane	1 : N	<i>NVARCHAR</i>	Kitap/Üye Eşleşmesi
Araç Filo	M : N (Ara Tablo)	<i>DECIMAL</i>	Aksesuar ve Envanter
Spor Salonu	Zaman Serisi	<i>DATETIME</i> / <i>BIGINT</i>	Giriş-Çıkış Analizi

Özet: Sistem Mimarisi ve Veri Kalitesi

Tasarladığımız modeller, veritabanı yönetiminin temel sütunlarını temsil eder:

Yapısal Taksonomi:

- **Ana Varlıklar:** Araç, Üye, Kitap (Sistemin sabitleri).
- **Hareketler:** Kiralama, Log (Sistemin dinamik değişkenleri).
- **Köprüler:** Ara tablo kullanımları (ilişkisel simetri).

Veri Entropisini Düşüren Kısıtlar:

- **PK/FK:** İlişkisel determinizm sağlar.
- **CHECK/UNIQUE:** Veri saflığını korur.
- **BIT/DATETIME:** Zaman ve durum uzayını yönetir.

Haftalık Kontrol Listesi: Neler Öğrendik?

Bu laboratuvar oturumunun sonunda aşağıdaki yetkinlikleri kazanmış olmalısınız:

- ☐ **CREATE DATABASE** ile izole ve temiz bir çalışma alanı kurabiliyorum.
- ☐ **PRIMARY KEY** (Tekillik) ve **IDENTITY** (Otomatik Artış) mekanizmalarını yönetebiliyorum.
- ☐ **CHECK** kısıtlaması ile veriyi fiziksel ve mantıksal sınırlarında tutabiliyorum.
- ☐ **FOREIGN KEY** ile tablolar arası referansal bütünlük sağlıyorum.

Vizyon

Veritabanı sadece veri saklamaz; kısıtlamalar (constraints) aracılığıyla verinin mantıksal tutarlılığını sağlar. Bir kısıt hatası, tüm sistemin entropisini artırarak kararsızlığa yol açar.

Gelecek Hafta:

Tasarladığımız bu yapıları SQL sorgularıyla (DML) canlandıracağız.
Seçme, Filtreleme ve Birleştirme (JOIN) işlemleri.

Dr. Öğr. Üyesi Hasan Oğuz

İstanbul Okan Üniversitesi

hasan.oguz@okan.edu.tr